

GCP Cloud Operations – Hands-On Homework & Live Defense

1. Purpose

The purpose of this homework is to validate hands-on GCP Cloud Operations experience. You will build a small production-like environment, monitor it, inject a realistic fault, diagnose and fix it, and document your work.

You may use an LLM or AI assistant. However, you must verify all output, understand what you deploy, and be ready to explain, modify, and troubleshoot the environment. A polished repository alone is not sufficient. The live defense is the primary assessment.

2. Sandbox & Safety Rules

- Candidates must use their personal free GCP trial account.
- Do not use production data, production credentials, or personal billing accounts.
- Do not delete the GCP project.
- Do not run `terraform destroy` before the live defense call. Keep the environment available for the live session.
- A budget alert has been configured. If you expect to exceed it, contact us first.

You may use your own GIT repository, but do not commit secrets, keys, or passwords.

3. Tasks

Task 1 — Infrastructure as Code with Terraform

Build the following using Terraform:

- VPC and subnet.
- Cloud Router and Cloud NAT.
- Firewall rules for health checks and internal traffic.
- Private GKE cluster or node pool with:
 - Workload Identity enabled.

- Autoscaling.
 - Appropriate node service account.
- Cloud SQL for PostgreSQL with private IP.
- Secret Manager secret for database credentials.
- Least-privilege service account for the application.

Remote state stored in a GCS bucket.

Requirements:

- Use variables, outputs, and a clear file/module structure.
- Run `terraform init`, `terraform plan`, and `terraform apply`.
- Capture raw outputs:
 - `terraform output -json`
 - `terraform state list`
 - `gcloud container clusters describe ...`

`gcloud sql instances describe ...`

Expected Output:

Submit the Terraform code and evidence that it was successfully applied in your GCP project.

Include:

- Terraform files and remote state configuration.
- `terraform init`, `plan`, `apply`, `output -json`, and `state list` outputs.
- `gcloud describe` outputs for GKE, Cloud SQL, VPC, NAT, firewall, service accounts, and Secret Manager.
- Screenshots of the GCP console showing the created resources.
- README section and architecture diagram.
- AI Usage Log for Task 1.

Do not destroy the environment before the live defense call. Provide the destroy command but do not run it yet.

Task 2 — GKE Deployment and Operations

Deploy a sample application to GKE with:

- Deployment, Service, Ingress, ConfigMap, and Secret.
- Horizontal Pod Autoscaler (HPA).
- PodDisruptionBudget (PDB).
- Readiness and liveness probes.
- Resource requests and limits.

Connection to Cloud SQL using Cloud SQL Auth Proxy or the Cloud SQL CSI driver.

Demonstrate:

- Rolling update.
- Rollback.
- Scale up/down.
- `kubectl get pods -A -o wide`
- `kubectl describe pod`

`kubectl logs`

Expected Output:

Submit the Kubernetes manifests and evidence that the application was successfully deployed and operated on GKE. Include:

- YAML files for Deployment, Service, Ingress, ConfigMap, Secret, HPA, PDB, and Cloud SQL connection.
- `kubectl` apply, get, describe, and logs outputs.
- Evidence of rolling update, rollback, and scale up/down.
- HPA configuration and, if possible, scaling behavior.
- Proof of successful Cloud SQL connectivity.
- Screenshots of the GKE console and application response.
- README section and updated architecture diagram.
- AI Usage Log for Task 2.

Do not delete the deployment or destroy the cluster before the live defense call.

Task 3 — Observability and Alerting

Create:

- A Cloud Monitoring dashboard showing:
 - GKE CPU/memory.
 - Pod restarts.
 - Cloud SQL connections.
 - Latency and error rate.
- Alert policies for:
 - CrashLoopBackOff or repeated pod restarts.
 - Node CPU above 80% for 5 minutes.
 - Cloud SQL connections above 80%.
 - Uptime check failure.

A log-based metric for application errors.

Expected Output:

Capture a screenshot or evidence of an alert firing and resolving. Submit evidence that you configured and validated observability and alerting for your GKE environment. Include:

- A Cloud Monitoring dashboard showing GKE CPU/memory, pod restarts, Cloud SQL connections, latency, and error rate.

- Four alert policies: CrashLoopBackOff/pod restarts, node CPU >80% for 5 minutes, Cloud SQL connections >80%, and uptime check failure.
- At least one notification channel configured and verified.
- A log-based metric for application errors, with evidence it captures data.
- An uptime check for the application endpoint, with an attached alert policy.
- Evidence of at least one alert firing and resolving, with a timeline.
- gcloud outputs and console screenshots for all monitoring resources.
- README section and updated architecture diagram.
- AI Usage Log for Task 3.

Do not delete the monitoring configuration before the live defense call.

Task 4 — Break / Fix / RCA

Inject **one** realistic operational fault into your environment. Choose exactly one from the approved list below. If you wish to propose your own fault, you must get it approved in advance.

Document:

- What you broke and why you chose it.
- Symptoms observed.
- Commands and logs used.
- Root cause.
- Fix applied.
- Preventive control.

Rollback or cleanup steps.

Approved Fault Menu — choose one:

1. Networking

- Remove Cloud NAT so GKE nodes cannot pull images.
- Add a firewall rule that blocks health check traffic.
- Break the private Google access path.

Misconfigure the Cloud SQL private IP route.

2. IAM / Security

- Remove the Workload Identity binding.
- Remove secretAccessor permission from the application service account.
- Use the wrong service account for the node pool.

Grant an overly broad role and then fix it to least privilege.

3. Kubernetes

- Set a bad readiness probe.
- Remove resource requests so HPA cannot scale.
- Use a wrong Secret reference.
- Set an insufficient memory limit causing OOMKilled.

- e. Break the Service selector so traffic does not reach pods.
- f. Misconfigure the Ingress.

Break the HPA metrics.

4. Database / Application

- a. Use the wrong Cloud SQL connection name.
- b. Remove the Cloud SQL Auth Proxy sidecar.
- c. Put a wrong password in Secret Manager.

Break a ConfigMap value the application needs.

5. Terraform / Config Drift

- a. Manually change a resource in the console and detect/reconcile drift.
- b. Introduce a wrong variable and trace the plan.

Break remote state configuration.

Do not choose: deleting a pod, restarting a deployment, scaling to zero, deleting the VPC/project, or any fault that destroys data.

Expected Output:

Submit a complete Break/Fix/RCA report for one fault from the approved menu. Include:

- The fault chosen, why, and the exact injection command/action with timestamps.
- Before and after evidence of the environment state.
- Symptoms observed, including alerts fired and screenshots.
- Diagnostic commands and raw outputs, with your reasoning.
- Root cause analysis with timeline and 5 Whys.
- The exact fix applied and proof of recovery.
- At least one preventive control implemented.
- Rollback and cleanup commands (do not run cleanup before the live defense).
- AI Usage Log for Task 4.

Do not destroy the environment before the live defense call. Keep the environment in a fixed, healthy state.

Task 5 — Automation Script

Write a Bash or Python health-check script that:

- Checks pod status, HPA, Cloud SQL connectivity, and application endpoint.
- Outputs JSON.

Exits non-zero on failure.

Show it running successfully and failing.

Expected Output:

The Bash or Python scripts.

Task 6 — Cleanup Readiness

Do **not** destroy the environment before the live defense.

Instead, provide:

- The terraform destroy command or cleanup script.
- Output of terraform plan -destroy or equivalent.

A short note on what will be removed.

4. Deliverables & Formats

1. Submit a Git repository or ZIP file containing:
 - a. Terraform code.
 - b. Kubernetes manifests.
 - c. Automation scripts.
 - d. README with setup and architecture explanation.
 - e. Architecture diagram.
 - f. Evidence folder with:
 - i. Raw command outputs.
 - ii. Screenshots of console, dashboard, alerts.
 - iii. Logs and RCA.
2. AI Usage Log.
3. AI Usage Log Template

Prompt / Question	Tool Used	Output Accepted?	What You Verified	What You Rejected / Changed